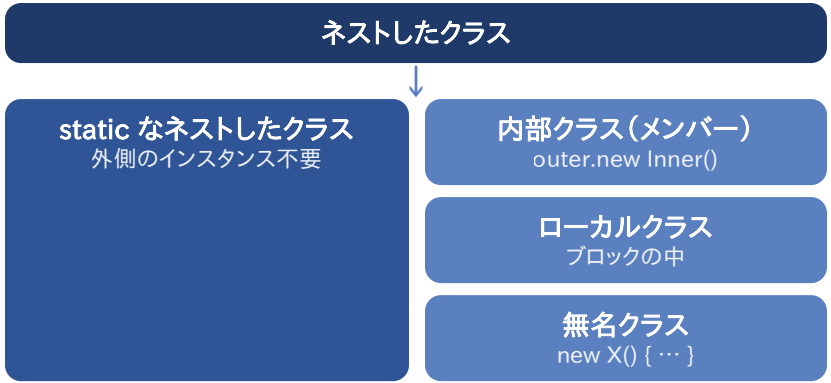
Java SE 17 Programmer II（1Z0-826-JPN）　試験内容 2 関数型インタフェースとラムダ式　#04

ネストしたクラス

内部クラス・static なネストしたクラス・ローカルクラス・無名クラス

各ページの下に、根拠にした資料（Oracle の試験内容チェックリスト、Java 言語仕様 JLS、JEP 395、The Java Tutorials の Nested Classes）を書いた。コードの結果は JDK 17（Eclipse Temurin 17.0.21）で実行して確かめた

全体像 クラスの中のクラス



右の 3 つ（static でないもの）をまとめて「内部クラス」と呼ぶ（JLS 8.1.3）。

getClass().getName()（バイナリ名）

Outer.Nested	Outer$Nested
Outer.Inner	Outer$Inner
最初の無名クラス	Outer$1

ネストしたクラスと外側は、お互いの private に届く（private の範囲はトップレベルクラスの本体）。

トップレベルクラスは public かなしだけ。private class Top は×。

出典：JLS 8.1.1・8.1.3・6.6.1・13.1／The Java Tutorials「Nested Classes」／JDK 17 での実行結果

第1章

内部クラスと static なネストしたクラス

外側のインスタンスを持つか、持たないか。

内部クラス（メンバー）

```
class Outer {
  private int x = 10;
  class Inner {
    int get() { return x * 2; }
  }
  public static void main(String[] a) {
    Outer o = new Outer();
    Outer.Inner in = o.new Inner();
    o.x = 7;
    in.get();        // 14（今の x を読む）
    new Inner();     // エラー：this がない
  }
}
```

名前が重なったとき

```
class Outer { int x = 1;
  class Inner { int x = 2;
    void show(int x) {
      x            // 3 引数
      this.x       // 2 Inner
      Outer.this.x // 1 外側
    } } }
```

> Java 16 から内部クラスも static メンバーを持てる。`class Inner { static int count; }` は Java 17 で〇。

出典：JLS 8.1.3・15.8.4・15.9.2、JEP 395／JDK 17 でのコンパイルと実行の結果

static なネストしたクラス

```
class Outer {
  private static int s = 1;
  private int i = 2;
  static class Nested {
    private int n = 5;
    int get() { return s + n; } // ○ 6
    int bad() { return i; }     // × インスタンス
  }
}
Outer.Nested a = new Outer.Nested(); // 外側なし
a.n  // Outer の中なら private でも読める
```

	内部クラス	static なネスト
作り方	outer.new Inner()	new Outer.Nested()
外側の static	○	○
外側のインスタンス	○	×
修飾子	public・protected・private・なし、final・abstract	

> インタフェースの中のクラスは暗黙に public static。`new Shape.Util()` で作れる。

出典：JLS 8.5・8.5.1・9.5・6.6.1／JDK 17 でのコンパイルと実行の結果

第2章

ローカルクラス

ブロックの中のクラスと、effectively final。

ローカルクラスと変数の捕捉

```
int base = 10;
class Adder {
  int add(int v) { return base + v; }
}
base = 20;   // これがあるとエラー
// local variables referenced from an inner
// class must be final or effectively final
```

修飾子

final・abstract	public・private・static
○	×

捕捉するもの	代入し直し
ローカル変数・引数	× effectively final
for の i（i++ する）	× 使えない
ループ内の int j = i;	○ 毎回別の変数
フィールド	○ 変えてよい
final int[] box の要素	○ 中身は変えられる

> Java 16 からローカルのレコード・インタフェース・列挙も書ける（暗黙に static）。

出典：JLS 4.12.4・8.1.3・14.3・15.27.2、JEP 395／JDK 17 でのコンパイルと実行の結果

第3章

無名クラス

名前のないクラスを、その場で 1 つだけ作る。

無名クラスの書き方と決まり

```
Greeter g = new Greeter() {     // インタフェース
  int count = 0;                // 状態を持てる
  public String greet(String n) {
    count++; return "Hi " + n + count;
  }
};
Animal a = new Animal("cat") {  // 親の引数
  String sound() { return name + ":nya"; }
};
```

決まり	結果
クラス 1 つ継承か、インタフェース 1 つ実装	どちらか一方
コンストラクタ `Greeter() { }`	エラー
インスタンス初期化子 `{ add("a"); }`	○
`Object o = new Object(){…}; o.hello()`	エラー
`var o = new Object(){…}; o.hello()`	○

出典：JLS 15.9.5・15.9.5.1・8.6・14.4.1／JDK 17 でのコンパイルと実行の結果

無名クラスとラムダ式の this

```
class Main {
  String name = "main";
  void run() {
    Runnable r1 = new Runnable() {
      String name = "anon";
      public void run() {
        System.out.print(this.name); // anon
      }
    };
    Runnable r2 = () ->
      System.out.println(this.name); // main
  }
}
```

	無名クラス	ラムダ式
this	自分自身	外側のインスタンス
新しいスコープ	作る	作らない
状態（フィールド）	持てる	持てない
実装できる型	クラス・任意のインタフェース	関数型インタフェースだけ

> new ArrayList<>() {{ add("a"); }} は ArrayList の子の無名クラス。
> getClass() != ArrayList.class。

出典：JLS 15.27.2・15.8.3・15.9.5／JDK 17 での実行結果